Section 06: Persistence & Checkpointing

Saving State Snapshots for Long-Running Agents

LangGraph v1.0.x (March 2026)

Mental Model: Why Persistence Matters

The Problem

- Agents run across **multiple API calls** (e.g., user messages)
- Without persistence, state is **lost** between invocations
- Conversations die, multi-step tasks fail if process crashes

The Solution

- **Checkpointing**: save state snapshot after each step
- **Thread IDs**: group related invocations (per user, per conversation)
- Combine them: resume exactly where you left off

What Is a Checkpointer?

Checkpointer Definition

- Object that **saves state** after each super-step
- Implements `get`, `put`, `list` methods
- Can be in-memory, SQLite, PostgreSQL, or custom
- Enables **time travel**: read state at any checkpoint

Checkpoint Timing

Checkpoints happen between graph steps, not inside nodes (that's too frequent).

MemorySaver: In-Memory Checkpointer

Development Setup

```python
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "user_123"}}
result = graph.invoke(
  {"messages": [HumanMessage("Hi")]},
  config=config
)
```

Development Only

MemorySaver loses data on process restart. Use only for prototyping.

Setting Up Checkpointing

Checkpointer Setup Pattern

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph
checkpointer = MemorySaver()
builder = StateGraph(MessagesState)
builder.add_node("agent", agent_node)
builder.add_edge(START, "agent")
graph = builder.compile(checkpointer=checkpointer)
```

Key Parameter

`checkpointer=` in `compile()` activates persistence.

Thread IDs: Organizing Conversations

Thread ID Pattern

```
user_id = "user_123"
thread_id = f"conv_{user_id}_{timestamp}"
config = {
  "configurable": {
    "thread_id": thread_id
  }
}
result = graph.invoke(
  {"messages": [HumanMessage("Hello")]},
  config=config
)
```

Thread Strategy

One thread per conversation. Group related messages under same thread_id.

Resuming Conversations

Resume from Last Checkpoint

```
config = {"configurable": {"thread_id": "conv_123"}}
result_1 = graph.invoke(
  {"messages": [HumanMessage("What's 2+2?")]},
  config=config
)
result_2 = graph.invoke(
  {"messages": [HumanMessage("And 3+3?")]},
  config=config
)
```

Automatic Resumption

LangGraph automatically loads last state from thread. New input appends to messages.

Getting State: Inspect Current Checkpoint

Retrieve Current State

```
config = {"configurable": {"thread_id": "conv_123"}}
state = graph.get_state(config)
print(state.values)
print(state.next)
```

Use Cases

Debugging, monitoring agent progress, checking decision points.

Updating State: Modify Externally

Manual State Update

```
config = {"configurable": {"thread_id": "conv_123"}}
new_state = {
  "messages": state.values["messages"] + [
    HumanMessage("Additional context")
  ]
}
graph.update_state(config, new_state)
result = graph.invoke(None, config)
```

Manual Edits

Useful for injecting human feedback or correcting mistakes.

State History: Time Travel

View All Checkpoints

```python
config = {"configurable": {"thread_id": "conv_123"}}
history = graph.get_state_history(config)
for i, state_snapshot in enumerate(history):
  print(f"Checkpoint {i}: {state_snapshot.values}")
  print(f"Next node: {state_snapshot.next}")
```

Debugging and Auditing

See entire execution trace. Useful for understanding why agent made a decision.

SqliteSaver: Persistent Local Storage

SQLite Checkpointer

```
from langgraph.checkpoint.sqlite import SqliteSaver
checkpointer = SqliteSaver(
  conn=sqlite3.connect(".langgraph.db")
)
graph = builder.compile(checkpointer=checkpointer)
```

Local Persistence

Data survives process restarts. Perfect for single-machine deployments.

PostgresSaver: Production-Grade Persistence

PostgreSQL Checkpointer

```
from langgraph.checkpoint.postgres import PostgresSaver
conn_string = "postgresql://user:pass@localhost/langgraph"
async with await asyncpg.create_pool(
  conn_string
) as pool:
  checkpointer = PostgresSaver(
    sync_connection=pool
  )
  graph = builder.compile(checkpointer=checkpointer)
```

Enterprise Ready

Distributed, scalable, supports concurrent access. Use for production.

TikZ Diagram: Checkpoint Flow

Pattern

Checkpoint after each node executes. Enables resumption from any point.

Super-Steps Explained

Super-Step Definition

- One **atomic unit** of graph execution
- Executes **all edges from START** to first blocking node
- Gets checkpointed after completion
- LangGraph waits for human/external input between super-steps

Concurrency

Super-steps are the natural unit for parallelization and interrupts.

Thread Management Patterns

Per-User Pattern

- One thread per user: `thread_id = f"user_user_id"`
- All conversations with that user share one thread
- Simpler, but context can get long

Per-Conversation Pattern

- One thread per conversation: `thread_id = f"conv_conv_id"`
- Separate contexts, easier to manage
- Recommended for most multi-turn systems

Common Pitfalls

Forgetting thread_id

- All invocations go to same (anonymous) thread
- Conversations pollute each other's state
- Solution: always pass `config={"configurable": {"thread_id": ...}}`

Stale State

- Reading state before all nodes finish
- Solution: let graph complete, then read from checkpoint

Large State Objects

- Storing huge arrays/files in checkpoints
- Solution: reference external storage, keep state lean

Summary: Persistence Cheat Sheet

Quick Reference

```python
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "conv_123"}}
result = graph.invoke(input_state, config=config)
state = graph.get_state(config)
history = graph.get_state_history(config)
graph.update_state(config, new_values)
```

Self-Check: Do You Understand?

1. Why is persistence critical for multi-turn conversations?
2. What's the difference between MemorySaver and SqliteSaver?
3. How do thread IDs organize checkpoints?
4. What happens when you invoke the same thread_id twice?
5. How do you inspect the state history?
6. What's a super-step and when do checkpoints happen?
7. When should you use per-user vs per-conversation threads?